

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a006669d746f38672.jsonl`  
- SHA-256: `7ff88342952d564546962f71db0e29e13f38fb0fff4f07e9ffed6b8d49c08313`  
- Source modified: `2026-06-22T15:04:42+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_f39953fd-98d`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:03:09.426Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: robustness-security

Severity: minor

Title: Tests never assert dispatcher video_id == worker video_id; an id-divergence regression would pass silently

File: tests/test_api_cloud_dispatch.py:39-66

Detail: The DB rows are written under the DISPATCHER's id `video\_id = compute\_video\_id(local\_video)` (the real MD5 of the fake fetched bytes, e.g. b708f3d1... for b'FAKEVIDEOBYTES'), while the bucket outputs path and result.outputs_uri are keyed by the WORKER/fake's `video\_id='vidCloud'`. These structurally differ in the fixture, yet every test passes because the DB write + query both use the dispatcher id and the CSV is read from result.outputs_uri (worker id) where the fake wrote it. In production the two ids SHOULD coincide (same MD5 of same bytes), but there is no guard in _maybe_dispatch_remote that they agree, and no test pins it. If the worker ever re-encodes/normalizes the input (changing its MD5) or fetches a different object, rows would be stored under a different id than the report/outputs are keyed by, and this would go completely undetected. The test suite's strongest invariant (id consistency end-to-end) is exactly the one it doesn't assert.

Fix: Either add a guard in _maybe_dispatch_remote that logs/fails when result.video_id (when non-empty) != the dispatcher's video_id, or at minimum add a test where the fake's video_id is set to the dispatcher's actual MD5 so the consistency is exercised, plus an assertion comparing the two.

2. assistant (2026-06-22T15:03:11.772Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:03:14.934Z)

---BRANCH---

feat/serving-api-cloud-dispatch

---HEAD---

304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF

4. assistant (2026-06-22T15:03:17.436Z)

This is a feature branch. Let me view the diff against origin/master and read the cited test file.

5. user (2026-06-22T15:03:18.606Z)

```
backend/main.py | 100 ++++++
tests/test_api_cloud_dispatch.py | 212 ++++++
2 files changed, 312 insertions(+)
```

6. user (2026-06-22T15:03:20.719Z)

```
1     """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7     dispatches; every other target runs the in-process segmenter byte-identically.
8     """
9     import csv
10    import json
11    import os
12
13    import pytest
14
15    pytest.importorskip("httpx")
16    from fastapi.testclient import TestClient
17
18    import backend.compute as compute_pkg
19    import backend.main as m
20    from backend.compute.cloud_run import CloudRunJobClient
21    from backend.config.models import AppConfig
22    from backend.infrastructure.database import Database
23    from backend.infrastructure.execution_policy import ExecutionPolicy
24    from backend.pipeline.validators.base import ValidationResult
25
26    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
27
28
29    def _pass():
30        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
31
32
33    class _InlineExecutor:
34        def submit(self, fn, *a, **k):
35            fn(*a, **k)
36
37
38    class _FakeCloudClient(CloudRunJobClient):
39        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
40        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
41
42        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
43            self.calls: list[list[str]] = []
44            self.video_id, self.returncode = video_id, returncode
45            self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
46                                                         (8.74, 11.38, 3, "unforced_error")]
47
48        def run_job(self, job_name, argv, *, region, project=None):
49            self.calls.append(list(argv))
50            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
51            done_uri = argv[argv.index("--done-uri") + 1]
52            if self.returncode == 0:
53                outputs = os.path.join(out_uri, "outputs", self.video_id)
```

```

54         os.makedirs(outputs, exist_ok=True)
55         with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
56             json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
57                       "status": "success"}, f)
58         with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
59 encoding="utf-8") as f:
60             w = csv.writer(f)
61             w.writerow(["start", "end", "shot_count", "ending_reason"])
62             for s, e, sc, er in self.rows:
63                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
64         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
65         with open(done_uri, "w", encoding="utf-8") as f:
66             json.dump({"video_id": self.video_id, "returncode": self.returncode,
67                       "segment_count": len(self.rows), "status": "success"}, f)
68         return "exec-1"
69
70     @pytest.fixture
71     def cloud_env(tmp_path, monkeypatch):
72         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
73 bucket read an
74 identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
75 hash+validate."""
76         db = Database(str(tmp_path / "t.db"))
77         bucket = tmp_path / "bucket"
78         bucket.mkdir()
79         cfg = AppConfig.model_validate({
80             "deployment": {"compute_target": "cloud_run"},
81             "storage": {"backend": "local", "output_root": str(bucket)},
82         })
83         monkeypatch.setattr(m, "db_instance", db)
84         monkeypatch.setattr(m, "global_config", cfg)
85         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
86         # A real gs:// input would be fetched to hash+validate; return a local fake so that
87 path is offline.
88         fake_local = tmp_path / "fetched.mp4"
89         fake_local.write_bytes(b"FAKEVIDEOBYTES")
90         monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
91 str(fake_local))
92         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
93 {}))
94         return db, bucket, monkeypatch
95
96     def _inject_fake(monkeypatch, fake):
97         """Make backend.compute.get_compute_backend (re-imported inside
98 _maybe_dispatch_remote) build a
99 real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
100 token."""
101         real = compute_pkg.get_compute_backend
102         monkeypatch.setattr(compute_pkg, "get_compute_backend",
103                             lambda config: real(config, run_client=fake, policy=_FAST,
104 token="tok"))
105
106     def _meta(row):
107         md = row.get("metadata")
108         return json.loads(md) if isinstance(md, str) else (md or {})
109
110     def test_cloud_dispatch_ingests_segments(cloud_env):
111         db, _bucket, monkeypatch = cloud_env
112         fake = _FakeCloudClient()
113         _inject_fake(monkeypatch, fake)

```

```

110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"
113         assert len(out["play_segments"]) == 2
114         rows = db.query_play_segments(out["video_id"], {})
115         assert len(rows) == 2
116         metas = [_meta(r) for r in rows]
117         assert all(md["source"] == "cloud_run" for md in metas)
118         assert {md.get("shot_count") for md in metas} == {5, 3}
119         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env
124         fake = _FakeCloudClient()
125         _inject_fake(monkeypatch, fake)
126
127         m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
128
129         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
130         argv = fake.calls[0]
131         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
132         assert argv[argv.index("--out-uri") + 1] == str(bucket)
133         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
134         joined = " ".join(argv)
135         assert "deployment.compute_target=local_gpu" in joined # container runs INLINE
(never re-dispatch)
136         assert "indexing.detector_impl=native" in joined
137
138
139     def test_cloud_failure_marks_failed_no_rows(cloud_env):
140         db, _bucket, monkeypatch = cloud_env
141         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
142         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
143         fake = _FakeCloudClient(returncode=1)
144         _inject_fake(monkeypatch, fake)
145
146         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
147         assert out["status"] == "failed"
148         assert "returncode 1" in out["message"]
149         assert out["play_segments"] == []
150         assert db.query_play_segments(out["video_id"], {}) == []
151
152
153     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
154         _db, _bucket, monkeypatch = cloud_env
155         fake = _FakeCloudClient()
156         _inject_fake(monkeypatch, fake)
157         local_path = str(tmp_path / "local_only.mp4")
158
159         out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
160         assert out["status"] == "failed"
161         assert "cloud input" in out["message"]
162         assert fake.calls == [] # never dispatched a local-only path
163
164
165     def test_default_off_runs_in_process(tmp_path, monkeypatch):
166         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
167         byte-identically (the gate is inert). No Cloud Run dispatch."""

```

```

168     db = Database(str(tmp_path / "t.db"))
169     cfg = AppConfig() # compute_target defaults to ""
170     monkeypatch.setattr(m, "db_instance", db)
171     monkeypatch.setattr(m, "global_config", cfg)
172     monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
173     {}))
174     video = tmp_path / "v.mp4"
175     video.write_bytes(b"hashable-bytes")
176     class _InProcSeg:
177         def __init__(self, db_, cfg_):
178             self.db = db_
179
180         def process_video(self, vid, path, *a, **k):
181             sid = self.db.add_play_segment(vid, 0.0, 5.0)
182             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
183 "metadata": {}}]
184     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
185     out = m._execute_processing(m.ProcessRequest(video_path=str(video),
186     segmenter="motion"))
187     assert out["status"] == "success"
188     assert len(out["play_segments"]) == 1
189     assert "cloud_run" not in out["message"]
190
191     def test_full_api_process_to_query_via_cloud(cloud_env):
192         """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
193     worker dispatches
194         to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
195         db, _bucket, monkeypatch = cloud_env
196         fake = _FakeCloudClient(video_id="vidCloud2")
197         _inject_fake(monkeypatch, fake)
198         client = TestClient(m.app)
199         r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
200     "wasb_hybrid"})
201         assert r.status_code == 202
202         job_id = r.json()["job_id"]
203
204         job = client.get(f"/api/jobs/{job_id}").json()
205         assert job["status"] == "done"
206         video_id = job["video_id"]
207         assert video_id
208
209         q = client.post("/api/query", json={"video_id": video_id})
210         assert q.status_code == 200
211         segs = q.json()["play_segments"]
212         assert len(segs) == 2
213         assert all(_meta(s)["source"] == "cloud_run" for s in segs)

```

7. assistant (2026-06-22T15:03:23.334Z)

Now let me read the production code in `backend/main.py` that this tests ? specifically `\_maybe\_dispatch\_remote` and `\_execute\_processing`.

8. user (2026-06-22T15:03:24.368Z)

```

1     import os
2     import sys
3     import argparse
4     import csv
5     import json
6     import logging

```

```

7     import shutil
8     import socket
9     import tempfile
10    import threading
11    import uuid
12    import webbrowser
13    from importlib.metadata import PackageNotFoundError, version as _pkg_version
14    from urllib.parse import quote
15    import uvicorn
16    from dotenv import load_dotenv
17
18    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
19    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
20    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
21    # stdlib-only import ? keeps this pre-pydantic line light.
22    from backend.utils.app_home import env_file_to_load, resolve_output_dir
23
24    load_dotenv(env_file_to_load())
25
26    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
27    logging.basicConfig(
28        level=logging.INFO,
29        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
30        datefmt='%H:%M:%S'
31    )
32    logger = logging.getLogger(__name__)
33    from fastapi import FastAPI, HTTPException, Request
34    from fastapi.middleware.cors import CORSMiddleware
35    from starlette.middleware.trustedhost import TrustedHostMiddleware
36    from pydantic import BaseModel
37    from typing import List, Dict, Any, Optional
38
39    from fastapi.staticfiles import StaticFiles
40    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
41    from backend.pipeline.validators.base import registry
42    from backend.infrastructure.database import Database
43    from backend.pipeline.segmenters.base import segmenter_registry
44    from backend.pipeline.stitcher.compiler import VideoCompiler
45    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
46    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
47    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
48    from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
49    from backend.utils.single_instance import acquire_lock # P0e: one instance per database
50    from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
51    from backend.config import ( # new typed config
52        load_config as load_app_config,
53        write_light_default_config, # P2b: batteries-included first-run config seeding
54        AppConfig,
55        StitchingConfig,
56        enforce_startup_checks,
57        StartupCheckError,
58    )
59    from backend.results import Failure # standardized error/result contract
60    from backend.reporting import emit_indexer_report
61    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
62    from backend import billing # M8: per-job cost ledger (USD internal / INR display)
63    from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-

```

```

OFF)
64
65     app = FastAPI(title="Sports Highlight Indexer API")
66
67
68     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
69         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
70         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
71         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
72         src = os.environ if env is None else env
73         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
74         origins = [o.strip() for o in raw.split(",") if o.strip()]
75         return origins or ["*"]
76
77
78     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
79     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
80     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
81     # lock it to its origin; the middleware is fixed at startup.
82     app.add_middleware(
83         CORSMiddleware,
84         allow_origins=_cors_origins_from_env(),
85         allow_credentials=False,
86         allow_methods=["*"],
87         allow_headers=["*"],
88     )
89
90
91     def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
92         """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
93         web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
94         the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
95         Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
96         Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
97         ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
98         deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
99         src = os.environ if env is None else env
100         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
101         hosts = [h.strip() for h in raw.split(",") if h.strip()]
102         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
103         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host (":::1:port"),
so IPv6
104         # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
105         return hosts or ["localhost", "127.0.0.1", "testserver"]
106
107
108     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
109     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect

```

```

110     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
111     app.add_middleware(
112         TrustedHostMiddleware,
113         allowed_hosts=_allowed_hosts_from_env(),
114         www_redirect=False,
115     )
116
117
118     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
119         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
120         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
121         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
122         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
123         if cli_host and cli_host.strip():
124             return cli_host.strip()
125         env = os.environ.get("RALLY_BIND_HOST")
126         if env and env.strip():
127             return env.strip()
128         sec = getattr(config, "security", None)
129         if getattr(sec, "edge_auth_enabled", False):
130             return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
131         return "127.0.0.1"
132
133
134     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
135
136     def _port_is_free(host: str, port: int) -> bool:
137         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
138         uvicorn surfaces a real bind failure loudly)."""
139         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
140             try:
141                 s.bind((host, port))
142                 return True
143             except OSError:
144                 return False
145
146
147     def _find_free_port(host: str) -> int:
148         """An OS-assigned free port on host."""
149         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
150             s.bind((host, 0))
151             return int(s.getsockname()[1])
152
153
154     def _resolve_app_port(host: str, port: int) -> int:
155         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
156         return port if _port_is_free(host, port) else _find_free_port(host)
157
158
159     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
160         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
161         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
162         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""

```

```

163     if os.environ.get("RALLY_NO_BROWSER"):
164         return False
165     import time as _time
166     import urllib.request
167     deadline = _time.monotonic() + timeout
168     while _time.monotonic() < deadline:
169         try:
170             with urllib.request.urlopen(health_url, timeout=2) as r:
171                 if getattr(r, "status", 200) == 200:
172                     break
173             except Exception:
174                 _time.sleep(0.3)
175         else:
176             return False
177         try:
178             webbrowser.open(url)
179         except Exception:
180             pass
181     return True
182
183
184     # Serves static frontend files directly (now under api/ per approved structure).
185     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
186     # an
187     # arbitrary directory finds the shipped assets instead of creating a stray
188     # ./backend/api/static.
189     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
190     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
191     try:
192         os.makedirs(_STATIC_DIR, exist_ok=True)
193     except OSError:
194         # The dir ships with the package (a no-op create); guard against a read-only install
195         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
196         # PermissionError.
197         pass
198     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
199
200     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
201     # snapshot
202     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
203     # if it's
204     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
205     # no SPA change
206     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
207     # is enough
208     # (no client-side path routing ? no history-fallback catch-all, which would also be a
209     # route-order hazard).
210     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
211     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
212     else _STATIC_DIR
213     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
214     if os.path.isdir(_UI_ASSETS_DIR):
215         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
216
217
218     @app.get("/", response_class=HTMLResponse)
219     def serve_index():
220         index_path = os.path.join(_UI_DIR, "index.html")
221         if os.path.exists(index_path):
222             with open(index_path, "r", encoding="utf-8") as f:
223                 return f.read()
224         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
225
226
227     def _bundled_ui_version() -> Optional[Dict[str, Any]]:

```

```

219         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
220         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
221         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
222         try:
223             with open(path, "r", encoding="utf-8") as f:
224                 return json.load(f)
225         except (OSError, ValueError):
226             return None
227
228     # Global variables initialized at startup
229     db_instance: Optional[Database] = None
230     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
231     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
232     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
233     _instance_lock: Optional[Any] = None
234
235     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
236     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
237     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
238     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
239     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
240     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
241     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
242     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
243         JobWaiting,
244         _arm_wake_timer,
245         _drain_due_jobs,
246         _get_executor,
247         _job_to_request,
248         _MAX_DRAIN_WAKE_SEC,
249         _maybe_record_job_cost,
250         _run_claimed_job,
251         _schedule_next_drain_if_waiting,
252     )
253
254     # Legacy thin wrapper for any remaining direct calls during transition.
255     # New code should import load_app_config / AppConfig from backend.config directly.
256     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
257         cfg = load_app_config(config_path)
258         return cfg.model_dump(mode="json")
259
260     # --- API Models ---
261     class ProcessRequest(BaseModel):
262         video_path: str
263         player_pool: Optional[List[str]] = None
264         max_frames: Optional[int] = None
265         segmenter: Optional[str] = None
266         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
267         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
268         locale: Optional[str] = None
269
270     class QueryRequest(BaseModel):
271         video_id: str
272         server: Optional[str] = None
273         receiver: Optional[str] = None
274         duration_min: Optional[float] = None

```

```

275         event_type: Optional[str] = None
276
277     class CompileRequest(BaseModel):
278         video_id: str
279         segment_ids: List[int]
280         output_filename: str
281         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
282         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
283         locale: Optional[str] = None
284
285     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287
288         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
290         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
291         """
292         if segments:
293             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294         else:
295             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
296
297
298     # --- API Endpoints ---
299     @app.get("/api/config")
300     def get_config():
301         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
302         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
303         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
304         # by tests/test_redact.py + the /api/config redaction test).
305         if global_config is None:
306             return {}
307         return redact_secrets(global_config.model_dump(mode="json"))
308
309     @app.get("/api/health")
310     def health():
311         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
312         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
313         confirm the engine is reachable and which detector is wired. Never raises."""
314         sport = getattr(global_config, "sport", None)
315         indexing = getattr(global_config, "indexing", None)
316         return {
317             "status": "ok",
318             "sport": sport,
319             "default_segmenter": getattr(indexing, "default_segmenter", None),
320         }
321
322
323     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
324     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
325     API_VERSION = 1
326
327
328     def _engine_version() -> str:
329         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""

```

```

330         try:
331             return _pkg_version("badminton-highlight-indexer")
332         except PackageNotFoundError:
333             return "unknown"
334
335
336     @app.get("/api/version")
337     def api_version():
338         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
339         P1)."""
340         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
341         _bundled_ui_version()}
342
343     @app.get("/api/capabilities")
344     def capabilities():
345         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
346         step from
347         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
348         never its
349         value. Best-effort + never raises (the wizard degrades gracefully)."""
350         indexing = getattr(global_config, "indexing", None)
351         deployment = getattr(global_config, "deployment", None)
352         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
353         ffprobe = shutil.which(ffprobe_bin())
354         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
355         wasb = getattr(indexing, "wasb", None)
356         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
357         or ""
358         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
359         NOT
360         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
361         negative on a
362         # real GPU box) and importing it would add seconds to this probe. The detector
363         healthcheck is the
364         # authority on whether GPU inference actually works; this is just the wizard's
365         first-order signal.
366         nvidia_smi = shutil.which("nvidia-smi")
367         return {
368             "api_version": API_VERSION,
369             "engine_version": _engine_version(),
370             "segmenters": sorted(segmenter_registry.list_available().keys()),
371             "default_segmenter": getattr(indexing, "default_segmenter", None),
372             "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
373             "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
374             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
375             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
376             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
377             "deployment": {
378                 "profile": getattr(deployment, "profile", "") or "",
379                 "compute_target": getattr(deployment, "compute_target", "") or "",
380             },
381         },
382     }
383
384     @app.get("/api/videos")
385     def list_videos(limit: Optional[int] = None):
386         """List indexed videos (most recent first) so the console survives a reload ?
387         previously only a
388         single-row get existed (PACKAGING_PLAN.md P1)."""
389         return {"videos": db_instance.list_videos(limit=limit)}
390
391
392     def _reels_dir() -> str:
393         return getattr(getattr(global_config, "output", None), "output_dir", "output") or

```

```

"output"
385
386
387     _REEL_EXTS = (".mp4", ".mov")
388
389
390     @app.get("/api/reels")
391     def list_reels():
392         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
393
394         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
395         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
396         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
397         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
398         base = _reels_dir()
399         reels = []
400         try:
401             entries = os.scandir(base)
402         except OSError:
403             return {"reels": []}
404         with entries:
405             for e in entries:
406                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
407                     try:
408                         st = e.stat()
409                     except OSError:
410                         continue
411                     reels.append({
412                         "filename": e.name,
413                         "size_bytes": st.st_size,
414                         "modified_at": st.st_mtime,
415                         "download_url": f"/api/reels/{quote(e.name)}",
416                     })
417         reels.sort(key=lambda r: r["modified_at"], reverse=True)
418         return {"reels": reels}
419
420
421     @app.get("/api/reels/{filename}")
422     def download_reel(filename: str):
423         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
424         Mirrors /api/highlights' safety (bare-name + containment checks)."""
425         try:
426             name = safe_output_filename(filename)
427         except ValueError as e:
428             raise HTTPException(status_code=400, detail=str(e)) from e
429         base = _reels_dir()
430         path = os.path.join(base, name)
431         try:
432             path = resolve_under_root(path, base)
433         except ValueError as e:
434             raise HTTPException(status_code=400, detail=str(e)) from e
435         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
436             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
437         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
438         return FileResponse(path, media_type=media, filename=name)
439
440
441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
int:

```

```

442         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
443         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
444         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
445         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
470
471     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
472                                play_wm: int, cand_wm: int, notify) -> Optional[Dict[str,
Any]]:
473         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
474         == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
475         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
476
477         Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
478         fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
479         for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
480         from backend.compute import ComputeJobSpec, get_compute_backend
481
482         compute = get_compute_backend(global_config)
483         if compute is None:
484             return None # default-OFF ? run the in-process segmenter below (unchanged)
485
486         storage_cfg = getattr(global_config, "storage", None)
487
488         def _failed(msg: str) -> Dict[str, Any]:
489             # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
490             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
491             return {"video_id": video_id, "status": "failed", "message": msg,
492                     "results": [r.model_dump() for r in val_results], "play_segments": []}
493

```

```

494         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
495         try:
496             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
497         except ValueError as e:
498             return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
499         if input_ref.backend == "local":
500             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
501                             "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503         if not out_root:
504             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505                             "? that's where the Cloud Run job writes the result.")
506
507         notify("dispatching (cloud_run)")
508         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509         spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510         try:
511             result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
512         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {})] if
int(s.get("id", 0)) > play_wm
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
the sync endpoint used to return (UI compatibility); the per-video observability
report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
(after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The

```

```

original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
571                     "status": "failed",
572                     "message": "Video failed validation checks.",
573                     "results": [r.model_dump() for r in val_results],
574                 }
575                 return response
576
577             # 2. Run segmenter & indexer
578             logger.info("[API] Video passed checks. Segmenting and indexing...")
579             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581             segmenter_cls = segmenter_registry.get(seg_name)
582             if not segmenter_cls:
583                 # Submit-time validation already 400s unknown names; this is the defensive
584                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585                 available = list(segmenter_registry.list_available().keys())
586                 response = {
587                     "video_id": video_id,
588                     "status": "failed",
589                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590                     "results": [r.model_dump() for r in val_results],
591                     "play_segments": [],
592                 }
593                 return response
594
595             logger.info(f"[API] Using segmenter: {seg_name}")
596             notify(f"segmenting ({seg_name})")
597             segmenter_actual = seg_name
598             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599             # if the run yields segments; otherwise drop the new partial rows and keep the
old.

```

```

600         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605         # they're dropped on the cloud path in this slice ? a documented follow-up.)
606         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607         if remote is not None:
608             segmentation_ran = True
609             response = remote
610             return response
611         segmenter = segmenter_cls(db_instance, global_config)
612         try:
613             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614         except Exception:
615             # A raising segmenter must not leave half-written rows: restore the pre-run
616             # state (same destroy-after-confirm contract as the Failure branch), then
let
617             # the job worker record the failure.
618             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619             raise
620         segmentation_ran = True
621
622         if isinstance(result, Failure):
623             logger.error(f"[API] Segmenter failed: {result.message}")
624             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625             response = {
626                 "video_id": video_id,
627                 "status": "failed",
628                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
629                 "results": [r.model_dump() for r in val_results],
630                 "play_segments": [],
631             }
632             return response
633
634         segments = result.value if hasattr(result, "value") else result
635         notify("finalizing")
636         _finalize_reingest(video_id, segments, play_wm, cand_wm)
637
638         response = {
639             "video_id": video_id,
640             "status": "success",
641             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
642             "results": [r.model_dump() for r in val_results],
643             "play_segments": segments,
644         }
645         return response
646     finally:
647         # Always emit the per-video observability report (next to the video, or to
648         # report.output_dir). Never raises. Surface the paths in the response.
649         paths = emit_indexer_report(
650             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651             val_results=val_results, val_context=val_context,
652             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654         )
655         if paths and isinstance(response, dict):

```

```

656         response["reports"] = paths
657
658
659     @app.post("/api/process", status_code=202)
660     def process_video(req: ProcessRequest, request: Request):
661         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
662
663         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
664         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
665         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
666         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
667         """
668         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
669         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
670         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
677         try:
678             validate_input_path(req.video_path, allowed_root=allowed_root)
679             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
680             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
681             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
682         except ValueError as e:
683             raise HTTPException(status_code=400, detail=str(e)) from e
684         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
685         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
686         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
687         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
688         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
689         # rejects a non-local URI as out-of-root.
690         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
691             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
692         if req.segmenter and not segmenter_registry.get(req.segmenter):
693             available = list(segmenter_registry.list_available().keys())
694             raise HTTPException(
695                 status_code=400,
696                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
697     )
698
699     job_id = uuid.uuid4().hex
700     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
701                                 req.player_pool, req.max_frames, owner_id=owner_id,
702                                 locale=req.locale):
703         raise HTTPException(status_code=500, detail="Failed to persist job record.")
704     _get_executor().submit(_drain_due_jobs)
705     return JSONResponse(
706         status_code=202,
707         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},

```

```

708         )
709
710
711     @app.get("/api/jobs/{job_id}/cost")
712     def get_job_cost(job_id: str):
713         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
714         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
715         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
716         cost = db_instance.get_job_cost(job_id)
717         if cost is None:
718             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
719         fx = global_config.billing.fx_inr_per_usd
720         usd = cost.get("cost_usd")
721         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
722         cost["fx_inr_per_usd"] = fx
723         return cost
724
725
726     @app.get("/api/videos/{video_id}/cost")
727     def get_video_cost(video_id: str):
728         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
729         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
730         agg = db_instance.get_video_cost(video_id)
731         fx = global_config.billing.fx_inr_per_usd
732         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
733         agg["fx_inr_per_usd"] = fx
734         return agg
735
736
737     @app.get("/api/jobs/{job_id}")
738     def get_job(job_id: str):
739         """Job state: {status, progress, result, reports}. ``status`` = execution state
740         (queued|running|done|failed); the pipeline verdict is result["status"]."""
741         job = db_instance.get_job(job_id)
742         if not job:
743             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
744         result = job.get("result")
745         return {
746             "job_id": job["id"],
747             "status": job["status"],
748             "progress": job.get("progress"),
749             "video_id": job.get("video_id"),
750             "error": job.get("error"),
751             "result": result,
752             "reports": (result or {}).get("reports"),
753             "created_at": job.get("created_at"),
754             "started_at": job.get("started_at"),
755             "finished_at": job.get("finished_at"),
756         }
757
758     # --- Chunked upload (B2, PR-2) -----
759     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
760     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
761     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
762     # sequential-part logic, and abort cleanup ? is unchanged.
763     from backend.api import uploads as uploads_api
764
765     app.include_router(uploads_api.router)
766

```

```

767
768 # --- Highlight download (B3, PR-2) -----
769
770 @app.get("/api/highlights/{video_id}/{filename}")
771 def download_highlight(video_id: str, filename: str):
772     """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
773     (which only writes into output_dir; the browser previously got back an unreachable
774     server-local path)."""
775     try:
776         name = safe_output_filename(filename)
777     except ValueError as e:
778         raise HTTPException(status_code=400, detail=str(e)) from e
779     if not db_instance.get_video(video_id):
780         raise HTTPException(status_code=404, detail="Indexed video record not found.")
781     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
782     path = os.path.join(output_dir, name)
783     try:
784         path = resolve_under_root(path, output_dir)
785     except ValueError as e:
786         raise HTTPException(status_code=400, detail=str(e)) from e
787     if not os.path.isfile(path):
788         raise HTTPException(status_code=404,
789                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
790     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
791     return FileResponse(path, media_type=media, filename=name)
792
793
794 @app.post("/api/query")
795 def query_play_segments(req: QueryRequest):
796     filters = {
797         "server": req.server,
798         "receiver": req.receiver,
799         "duration_min": req.duration_min,
800         "event_type": req.event_type
801     }
802     segments = db_instance.query_play_segments(req.video_id, filters)
803     return {"play_segments": segments}
804
805 @app.post("/api/compile")
806 def compile_highlights(req: CompileRequest):
807     video = db_instance.get_video(req.video_id)
808     if not video:
809         raise HTTPException(status_code=404, detail="Indexed video record not found.")
810
811     # Retrieve matching play segments from db
812     all_segments = db_instance.query_play_segments(req.video_id, {})
813     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
814
815     if not matched_segments:
816         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
817
818     # Reject path traversal / absolute output names (os.path.join would otherwise let an
819     # absolute or '..'-laden output_filename write anywhere on disk).
820     try:
821         out_name = safe_output_filename(req.output_filename)
822     except ValueError as e:
823         raise HTTPException(status_code=400, detail=str(e)) from e
824
825     # The configured shot-count floor (stitching.min_shot_count) applies on the API
826     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count

```

```

827 # metadata are exempt: the floor filters known counts only, so explicitly
828 # requested manual/legacy segments can't silently vanish under the default floor.
829 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
830 kept = []
831 for s in matched_segments:
832     meta = s.get("metadata") or {}
833     sc = meta.get("shot_count") if isinstance(meta, dict) else None
834     try:
835         if sc is not None and int(sc) < stitching.min_shot_count:
836             continue
837     except (TypeError, ValueError):
838         pass # unparseable count -> treat as unknown, keep
839     kept.append(s)
840 if len(kept) < len(matched_segments):
841     logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
842               f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
843     if not kept:
844         raise HTTPException(status_code=400,
845                             detail=f"All {len(matched_segments)} matching segments fall
below "
846                                   f"stitching.min_shot_count={stitching.min_shot_count}.")
847
848     # Extract start/end time pairs
849     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
850
851     # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
852     # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
853     # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
854     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
855     localized_watermark = localize_watermark(stitching.watermark, req.locale)
856     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
857                             end_padding=stitching.end_padding,
watermark=localized_watermark)
858     try:
859         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
860         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
861         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
862         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
863         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
864         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
865         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
866         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
867         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
868         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
869         return {"status": "success", "filepath": out_path, "download_url": download_url}
870     except Exception as e:
871         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
872
873     # --- CLI Debug Utility & Orchestration ---

```

```

874 def run_cli_diagnose_clip(video_path: str, timestamp: float):
875     """CLI debugging utility to examine segment properties around a timestamp."""
876     print("=" * 60)
877     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
878     print("=" * 60)
879
880     cfg = load_config() # legacy wrapper still works, returns dict for now
881
882     # 1. Validation check diagnostics
883     print("[DIAGNOSTIC] Running validation framework...")
884     results = registry.run_all(video_path, cfg)
885     for r in results:
886         status_symbol = "[PASS]" if r.passed else "[FAIL]"
887         print(f" {status_symbol} {r.validator_name}: {r.message}")
888         if r.details:
889             print(f"         Details: {r.details}")
890
891     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
892     print("-" * 60)
893     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
894     import cv2
895     cap = cv2.VideoCapture(video_path)
896     if not cap.isOpened():
897         print("[FAIL] OpenCV could not open video.")
898         return
899
900     fps = cap.get(cv2.CAP_PROP_FPS)
901     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
902
903     start_frame = max(0, int((timestamp - 3.0) * fps))
904     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
905
906     # Measure frame-by-frame changes in sample region
907     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
908     prev_gray = None
909     motions = []
910
911     for _ in range(start_frame, end_frame):
912         ret, frame = cap.read()
913         if not ret:
914             break
915         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
916         gray = cv2.GaussianBlur(gray, (21, 21), 0)
917
918         if prev_gray is not None:
919             diff = cv2.absdiff(prev_gray, gray)
920             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
921             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
922 thresh.shape[1])
923             motions.append(motion_ratio)
924             prev_gray = gray
925
926     cap.release()
927
928     if motions:
929         avg_motion = sum(motions) / len(motions)
930         # Support both legacy dict (from wrapper) and future direct model
931         if hasattr(cfg, "indexing"):
932             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
933         else:
934             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
935 ignore[attr-defined]
936         print(f" Sample Frame Count: {len(motions)}")
937         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
938 (threshold: {threshold})")

```

```

936         if avg_motion > threshold:
937             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
938         else:
939             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
940     else:
941         print(" [ERROR] No visual frames were extracted in the sample range.")
942
943     print("=" * 60)
944
945
946     def _enforce_startup_or_exit(config) -> None:
947         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
948         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
949         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
950         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
951         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
952         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
953         fail-fast wiring + the default-OFF no-op are unit-testable."""
954         try:
955             enforce_startup_checks(config, logger=logger, include_exposure=True)
956         except StartupCheckError as e:
957             logger.error("[STARTUP] refusing to start: %s", e)
958             sys.exit(1)
959
960
961     def main():
962         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
963         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
964         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
965         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
966         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
967         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
968         parser.add_argument("--app", action="store_true",
969                             help="Launch the app: start the local server (localhost-only)
and open the "
970                                 "bundled UI in your browser. Falls back to a free port if
--port is busy.")
971         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
972         parser.add_argument("--host", default=None,
973                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
974                                 "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
975         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
976
977         available_segmenters = list(segmenter_registry.list_available().keys())
978         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
979         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")

```

```

980     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
981     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
982
983     args = parser.parse_args()
984
985     if args.setup:
986         # Invoke the diagnostics/bootstrap script (renamed from the old root
987         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
988         # build system). Resolve its path from this file so `indexer --setup`
989         # works regardless of the caller's cwd.
990         import subprocess
991         doctor_path = os.path.join(
992             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
993             "scripts", "doctor.py",
994         )
995         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
996         subprocess.run([sys.executable, doctor_path])
997         return
998
999     if args.trim_start is not None and args.trim_end is not None:
1000         if not args.video_path:
1001             print("[ERROR] Please provide --video-path to extract a segment.")
1002             return
1003
1004         # Determine output path
1005         out_filename = args.trim_output or "trimmed_segment.mp4"
1006         if os.path.isabs(out_filename):
1007             out_path = out_filename
1008             out_dir = os.path.dirname(out_path)
1009             out_name = os.path.basename(out_path)
1010         else:
1011             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
1012             out_path = os.path.join(out_dir, out_filename)
1013             out_name = out_filename
1014
1015             os.makedirs(out_dir, exist_ok=True)
1016             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
1017
1018             # global_config isn't initialized this early; load the typed config so the trim
1019             # clip carries the same configured stitching paddings + watermark as a real
1020             # highlight (watermark default-on).
1021             stitching = load_app_config().stitching
1022             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
1023                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
1024             try:
1025                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
1026                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1027             except Exception as e:
1028                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
1029             return
1030
1031     if args.debug_clip is not None:
1032         if not args.video_path:
1033             print("[ERROR] Please provide --video-path to debug a specific clip.")
1034             return
1035         run_cli_diagnose_clip(args.video_path, args.debug_clip)
1036         return
1037
1038     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly

```

```

`--app`
1039      # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1040      # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1041      # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1042      # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1043      # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1044      # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1045      if args.app:
1046          seeded_path, seeded = write_light_default_config()
1047          if seeded:
1048              print(f"[APP] First run: seeded a truly-local default config at
{seeded_path} "
1049                  f"(detector 'motion' ? offline, no API key or GPU needed). "
1050                  f"Use the in-app setup to switch to Gemini.")
1051
1052      # Initialize Global Config and DB
1053      global global_config, db_instance
1054      global_config = load_app_config() # returns typed AppConfig
1055      _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
1056
1057      # The CLI --output-dir overrides the configured output directory. It now lives
1058      # on the typed model (OutputConfig.output_dir), so every consumer ? including
1059      # /api/compile ? reads the same value instead of a write-only runtime hack.
1060      # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1061      # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1062      global_config.output.output_dir = resolve_output_dir(args.output_dir)
1063      os.makedirs(global_config.output.output_dir, exist_ok=True)
1064
1065      db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
1066
1067      # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1068      # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1069      # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1070      # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1071      global _instance_lock
1072      try:
1073          _instance_lock = acquire_lock(db_path + ".lock")
1074      except OSError as exc:
1075          # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1076          # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1077          # legitimate single instance).
1078          logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
1079                        "WITHOUT the second-instance guard.", db_path, exc)
1080          _instance_lock = None
1081      else:
1082          if _instance_lock is None:
1083              print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1084                  f"start ? a second instance would corrupt in-flight job state. Stop

```

```

the other one, or "
1085             f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1086             sys.exit(1)
1087
1088         db_instance = Database(db_path)
1089
1090         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1091         # the executor is in-process. Mark them failed so pollers see a terminal state.
1092         swept = db_instance.fail_stale_jobs()
1093         if swept:
1094             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1095
1096         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1097         if args.video_path and not args.server and not args.app:
1098             print(f"[CLI] Processing video file: {args.video_path}")
1099             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1100             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1101             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1102             storage_cfg = getattr(global_config, "storage", None)
1103             try:
1104                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1105             except ValueError as e:
1106                 print(f"[FAIL] {e}")
1107                 return
1108             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1109                 print(f"[FAIL] Video file not found: {args.video_path}")
1110                 return
1111             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1112
1113             video_id = compute_video_id(local_video)
1114             was_reingest = db_instance.get_video(video_id) is not None
1115
1116             # Validate (with-context variant surfaces ffprobe_meta for the report)
1117             print("[CLI] Validating...")
1118             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1119             failures = [r for r in val_results if not r.passed]
1120
1121             # Save video status
1122             status = "failed" if failures else "processed"
1123             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1124
1125             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1126             segmenter_actual = None
1127             segmentation_ran = False
1128             try:
1129                 print("\n" + "="*40)
1130                 print("VALIDATION SUMMARY")
1131                 print("="*40)
1132                 for r in val_results:
1133                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
1134                     print(f"{status_symbol} {r.validator_name}: {r.message}")
1135                 print("="*40 + "\n")
1136
1137             if failures:
1138                 print("[FAIL] Video failed checks. Indexing halted.")
1139                 return

```

```

1140
1141     # Index
1142     segmenter_cls = segmenter_registry.get(seg_name)
1143     if not segmenter_cls:
1144         print(f"[FAIL] Segmenter '{seg_name}' not found.")
1145         return
1146
1147     print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1148     segmenter_actual = seg_name
1149     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1150     segmenter = segmenter_cls(db_instance, global_config)
1151     result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
1152     segmentation_ran = True
1153
1154     if isinstance(result, Failure):
1155         print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1156         print("[FAIL] Indexing halted for video.")
1157         segments = []
1158     else:
1159         segments = result.value if hasattr(result, "value") else result
1160         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
1161         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1162         _finalize_reingest(video_id, segments, play_wm, cand_wm)
1163     finally:
1164         # Always emit the per-video observability report. Never raises.
1165         paths = emit_indexer_report(
1166             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1167             val_results=val_results, val_context=val_context,
1168             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
1169             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
1170         )
1171     if paths:
1172         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1173         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1174         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1175     return
1176
1177     # Start Local Server Mode
1178     if args.app or args.server or not args.video_path:
1179         if args.app:
1180             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1181             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1182             bind_host = "127.0.0.1"
1183             port = _resolve_app_port(bind_host, args.port)
1184             if port != args.port:
1185                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1186             url = f"http://{bind_host}:{port}/"
1187             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1188             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1189             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1190             print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1191                   f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1192             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1193                             daemon=True).start()
1194         else:

```

```

1195         bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1196         port = args.port
1197         print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1198         if bind_host not in ("127.0.0.1", "localhost", "::1"):
1199             print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1200                 f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1201                 f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1202                 f"loopback-only and will 400 forwarded requests otherwise).")
1203         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1204         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1205         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1206         _get_executor().submit(_drain_due_jobs)
1207         uvicorn.run(app, host=bind_host, port=port)
1208
1209     if __name__ == "__main__":
1210         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1211         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1212         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1213         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1214         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1215         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1216         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1217         import backend.main as _bm
1218
1219         _bm.main()
1220

```

9. assistant (2026-06-22T15:03:28.488Z)

Now I understand the production code. Let me verify the finding's central claim about the worker's `result.video\_id` and whether `\_maybe\_dispatch\_remote` uses it. The key questions:

1. Does the dispatcher use its own `video\_id` (computed locally) for both DB writes AND the outputs path?
2. Does it use `result.outputs\_uri` (worker-keyed) for reading the CSV?
3. Is there any guard that the two ids agree?

Let me look at the compute backend to understand `result.outputs\_uri` and `result.video\_id`.

10. user (2026-06-22T15:03:29.948Z)

```

__init__.py
__pycache__
base.py
cloud_run.py
---
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__init__.py

```

11. assistant (2026-06-22T15:03:32.920Z)

These compute files aren't in the diff (the diff only touches `backend/main.py` and the test). So they're pre-existing. Let me read them to understand `run\_infer`, `outputs\_uri`, and `video\_id`.

12. user (2026-06-22T15:03:33.084Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
```

```

48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

13. user (2026-06-22T15:03:35.595Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll.
11         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12            ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14         The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17         The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19         owner-verified on the M7 real run.
20         """
21
22         from __future__ import annotations
23
24         import json
25         import logging
26         import os
27         import tempfile
28         import uuid
29         from abc import ABC, abstractmethod
30         from typing import Any, List, Optional
31
32         from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33         from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34         from backend.storage import fetch, parse_uri
35         from backend.storage import get_storage
36
37         LOG = logging.getLogger("compute.cloud_run")
38
39         # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40         # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41         _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (

```

```

42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy

```

```

96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118
119         video_id = str(marker.get("video_id", ""))
120         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
121         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
122         rc = int(marker.get("returncode", 1))
123         if not video_id:
124             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
125                         "video_id) ? treating as failure", self._job_name)
126             rc = rc or 1
127         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
128         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
129         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
130         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
131
132     # --- storage-backed completion signal (bucket-poll) ---
133     def _read_marker(self, done_uri: str) -> Optional[dict]:
134         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
135         ref = parse_uri(done_uri)
136         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
137             return None
138         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
139
140     def _read_json(self, uri: str) -> dict:
141         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
142         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
143         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
144         try:

```

```

147         ref = parse_uri(uri)
148         dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149         local = fetch(uri, dst, config=self._storage_config)
150         with open(local, "r", encoding="utf-8") as f:
151             data: Any = json.load(f)
152         return data if isinstance(data, dict) else {}
153     except Exception:
154         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155         return {}
156

```

14. assistant (2026-06-22T15:03:42.074Z)

Now I have the full picture. The finding is structurally accurate about the mechanics. Let me confirm the exact discrepancy in the test fixture and trace the data flow precisely.

```

In `_maybe_dispatch_remote` (main.py:520):
- `_ingest_remote_segments(video_id, result.outputs_uri, storage_dict)` ? passes the
**dispatcher's** `video_id` (from `_execute_processing` line 549:
`compute_video_id(local_video)`) as the DB key, but reads the CSV from `result.outputs_uri` (the
**worker's** id-keyed path).

```

Let me verify the fixture's actual id values to confirm they structurally differ.

15. user (2026-06-22T15:03:43.696Z)

```
dispatcher video_id (MD5 of FAKEVIDEOBYTES): b708f3d1de52d89c6d374c6228a17178
```

16. assistant (2026-06-22T15:03:51.387Z)

Confirmed. The dispatcher's `video\_id` is `b708f3d1...` (MD5 of `b'FAKEVIDEOBYTES'`), while the fake worker's `video\_id` is `vidCloud` (or `vidCloud2`). These are structurally different in the fixture, exactly as the finding states.

Now let me verify the data flow precisely to confirm the finding's claim that the divergence is masked:

1. **DB write** (`\_ingest\_remote\_segments` line 466): `db\_instance.add\_play\_segment(video\_id, ...)` ? uses the **dispatcher** id `b708f3d1...`.
2. **DB query** (test line 114, and main.py line 522): `db.query\_play\_segments(out["video\_id"], {})` ? `out["video\_id"]` is the **dispatcher** id. Both write and query use the dispatcher id ? consistent regardless of worker id.
3. **CSV read** (`\_ingest\_remote\_segments` line 447): reads from `result.outputs\_uri` which the worker wrote at `outputs/vidCloud/` ? the fake wrote the CSV there.

So the dispatcher id and worker id never need to match for the tests to pass. The finding's mechanics hold.

Now let me check whether the worker's `video\_id` is used anywhere in `\_maybe\_dispatch\_remote` at all (to confirm there's no guard).

17. user (2026-06-22T15:03:52.711Z)

```

520:     _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
802:     segments = db_instance.query_play_segments(req.video_id, filters)
807:     video = db_instance.get_video(req.video_id)
812:     all_segments = db_instance.query_play_segments(req.video_id, {})
868:     download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"

```

18. assistant (2026-06-22T15:03:57.139Z)

Confirmed: `result.video\_id` is **never read** in `\_maybe\_dispatch\_remote`. Only `result.outputs\_uri` is consumed. There is no guard comparing `result.video\_id` to the

dispatcher's `video\_id`. The finding's core technical claims all hold up.

Now let me assess ****severity and real-world impact**** adversarially. The question is whether this is a genuine defect worth fixing or a nit.

Let me check the worker side to understand whether the production invariant (worker id == dispatcher id) is actually robust, and whether re-encoding could realistically change the MD5.

19. user (2026-06-22T15:03:58.999Z)

```
backend/api/job_worker.py
backend/worker/__init__.py
backend/worker/__main__.py
---worker module---
__init__.py
__main__.py
__pycache__
backend/api/uploads.py
backend/eval/golden_fixtures.py
backend/main.py
backend/utils/hashing.py
backend/utils/workspace.py
backend/worker/__main__.py
```

20. user (2026-06-22T15:04:02.480Z)

```
42:from backend.utils.hashing import compute_video_id
88:def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
91:     "video_id": video_id,
120:     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
137:     logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
138:                 result.video_id, result.returncode, result.outputs_uri)
139:     return result.returncode
142:def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count: int,
147:     marker = {"video_id": video_id, "returncode": returncode,
155:     logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
183:     video_id = compute_video_id(local_video)
189:     db.add_video(video_id, local_video, "failed" if failures else "processed",
208:     result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
222:     "infer produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s).
"
227:     video_id, segmenter_actual, detector_impl,
232:     db=db, video_id=video_id, video_path=local_video, config=config,
238:     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
239:     out_local = os.path.join(scratch, "outputs", video_id)
242:     _write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))
247:     uri = f"{out_prefix}/outputs/{video_id}/{fn}"
255:     # M6: a remote dispatcher passes --done-uri; write a completion marker so its bucket-
poll
256:     # terminates (carries video_id ? the dispatcher's output path + the worker rc). DEFAULT-
OFF:
257:     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a failed
container
260:     _write_done_marker(args.done_uri, video_id, 0, len(segments), status, storage_cfg,
scratch)
417:     video_id = compute_video_id(local_video)
425:     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
455:     rc = subprocess.run(cmd).returncode
483:     pi.add_argument("--out-uri", required=True, help="output prefix (outputs/<video_id>/? is
appended)")
489:     help="M6: storage URI for a completion MARKER (video_id + rc) written on
a "
```

21. assistant (2026-06-22T15:04:05.512Z)

Now let me look at the worker's input acquisition (line ~183) to assess whether the worker could compute a *different* MD5 than the dispatcher ? the crux of whether this is a real risk.

22. user (2026-06-22T15:04:06.555Z)

```
170         return _dispatch_remote(compute, args)
171
172     storage_cfg = config.storage.model_dump()
173
174     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
175     os.makedirs(scratch, exist_ok=True)
176     config.output.output_dir = scratch
177
178     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
179     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
180     print(f"[worker] pulled {args.video_uri} -> {local_video}")
181
182     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
183     video_id = compute_video_id(local_video)
184
185     # 3. VALIDATE (same registry as the main CLI).
186     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
187     failures = [r for r in val_results if not r.passed]
188     db = Database(os.path.join(scratch, config.output.db_name))
189     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
190
191     segments: List[dict] = []
192     segmenter_actual = None
193     segmentation_ran = False
194     status = "failed"
195
196     try:
197         if failures:
198             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
```

23. assistant (2026-06-22T15:04:23.438Z)

I now have the complete picture. The worker fetches the input bytes from the bucket and computes the MD5 of the *fetched* bytes (line 179-183), while the dispatcher computes the MD5 of *its* locally-acquired bytes. In production both fetch the same `gs://` object, so the MD5s coincide ? but this is an implicit coupling, never asserted.

Let me weigh the finding carefully:

****What holds up (the mechanics are correct):****

- The dispatcher keys DB writes by its own `video\_id`, while outputs path/CSV are keyed by the worker's `video\_id`. ?
- In the fixture these structurally differ (`b708f3d1...` vs `vidCloud`). ?
- All tests pass because write+query both use the dispatcher id, and the CSV read uses `result.outputs\_uri`. ?
- `result.video\_id` is never read in `\_maybe\_dispatch\_remote`; no guard exists. ?
- An id-divergence regression (worker re-encodes/normalizes, or fetches a different object) would store rows under one id but key outputs under another, undetected by these tests. ?

****Adversarial counter-weighing (does it matter?):****

- The dispatcher already verifies the input resolves to a non-local backend (`gs://`) and passes that *same URI* to the worker. The worker fetches the *same object*. So in production the two MD5s coincide by construction ? the divergence is genuinely unreachable today.

- ``compute_video_id`` is whole-file MD5; neither side re-encodes. The "worker re-encodes/normalizes" scenario is hypothetical future code, not present.
- The actual functional consequence if ids ever diverged: rows stored under the dispatcher id (which is what ``/api/query`` and ``/api/compile`` use via ``req.video_id`` / ``out["video_id"]``), but the CSV was read from the worker-id path. Since ``_ingest_remote_segments`` reads from ``result.outputs_uri`` (worker path, where the data actually is) and writes under the dispatcher id (where query reads), the *user-facing flow would actually still work* ? the rows land where query looks. The real latent risk is narrower: idempotency/dedup keyed on `video_id`, the `report.json`'s `video_id` field mismatching the DB rows, and cache/path confusion.

This is a real test-coverage gap, correctly diagnosed: the suite's strongest invariant (end-to-end id consistency) is the one not asserted, and the fixture obscures it by using different ids. That said, it is not a production bug today (the divergence is unreachable by construction), and the proposed "guard that fails the job" could even introduce a new failure mode. The accurate characterization is a **minor robustness/test-quality nit** ? worth a cheap test that pins the ids to match, low value to add a hard production guard.

The severity claim of "minor" is appropriate. It is a real, accurately-described gap, not a false alarm ? but it sits at the boundary of minor/nit. The proposed fix (pin the fake's ``video_id`` to the dispatcher MD5 and assert consistency) is reasonable and low-cost; the alternative (a hard guard that fails the job) is more debatable.

24. user (2026-06-22T15:04:33.429Z)

Structured output provided successfully

25. assistant (2026-06-22T15:04:42.721Z)

Verified the finding against the actual code in ``C:/Users/avidu/Projects/bhi-cloudrun``. It holds up as a real but minor (test-quality/robustness) gap.

Key files:

- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:520`` ? ``_ingest_remote_segments(video_id, result.outputs_uri, ...)`` passes the **dispatcher's** ``video_id`` (DB key) but reads from the **worker's** ``result.outputs_uri``; ``result.video_id`` is never consumed, so no consistency guard exists.
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:549`` ? `dispatcher id = `compute_video_id(local_video)``.
- ``C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/cloud_run.py:129`` ? outputs path keyed by the worker/marker ``video_id``.
- ``C:/Users/avidu/Projects/bhi-cloudrun/tests/test_api_cloud_dispatch.py:42`` ? fake worker ``video_id="vidCloud"``, structurally different from the dispatcher MD5 ``b708f3d1...`` (I computed it), yet every test passes.

Conclusion: ``is_real=true``, ``severity=minor``. The mechanics are exactly as described ? the suite's strongest invariant (end-to-end id consistency) is the one not asserted, and the fixture's mismatched ids obscure it. I dropped it to minor (not major) because the divergence is unreachable in production today: the dispatcher passes the same ``gs://`` URI to the worker, both fetch the same bytes and compute whole-file MD5, so the ids coincide by construction; the "worker re-encodes" path is hypothetical. The cheap half of the proposed fix (pin the fake's ``video_id`` to the dispatcher MD5 + assert) is worthwhile; the hard production guard is more debatable.